

So far we have focused on two types of functional forms

- A SINGLE input with a linear relationship:

$$\mu_n = \beta_0 + \beta_1 x_n$$

- A basis function applied to a SINGLE input:

$$\mu_n = \sum_{j=0}^J \left(\beta_j \phi_j(x_n) \right)$$

We have discussed a general case with D inputs...

$$\mu_n = \sum_{d=0}^D (\beta_d x_{n,d}) = \mathbf{x}_{n,:} \boldsymbol{\beta}$$

We have discussed a general case with D inputs...

$$\mu_n = \sum_{d=0}^D (\beta_d x_{n,d}) = \mathbf{x}_{n,:} \boldsymbol{\beta}$$

Trends from non-linear features derived from BASIS functions are also be calculated from inner products!!!

$$\mu_n = \sum_{j=0}^J (\beta_j \phi_j(x_n)) = \boldsymbol{\phi}_{n,:} \boldsymbol{\beta}$$

Let's specifically consider a model with 2 inputs x_1 and x_2

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

Let's now consider a case with 2 continuous inputs, x_1 and x_2

- Each input has a separate column in the training data set.
- We have to consider the kinds of features of features we will derive from these inputs!

Start simple...LINEAR ADDITIVE features!!

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

Do we calculate the trend differently now that we have a second input?

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

Do we calculate the trend differently now that we have a second input?

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

- NO!!!!
- The trend is still calculated from the DESIGN MATRIX using matrix multiplication!

$$\boldsymbol{\mu} = \mathbf{X}\boldsymbol{\beta}$$

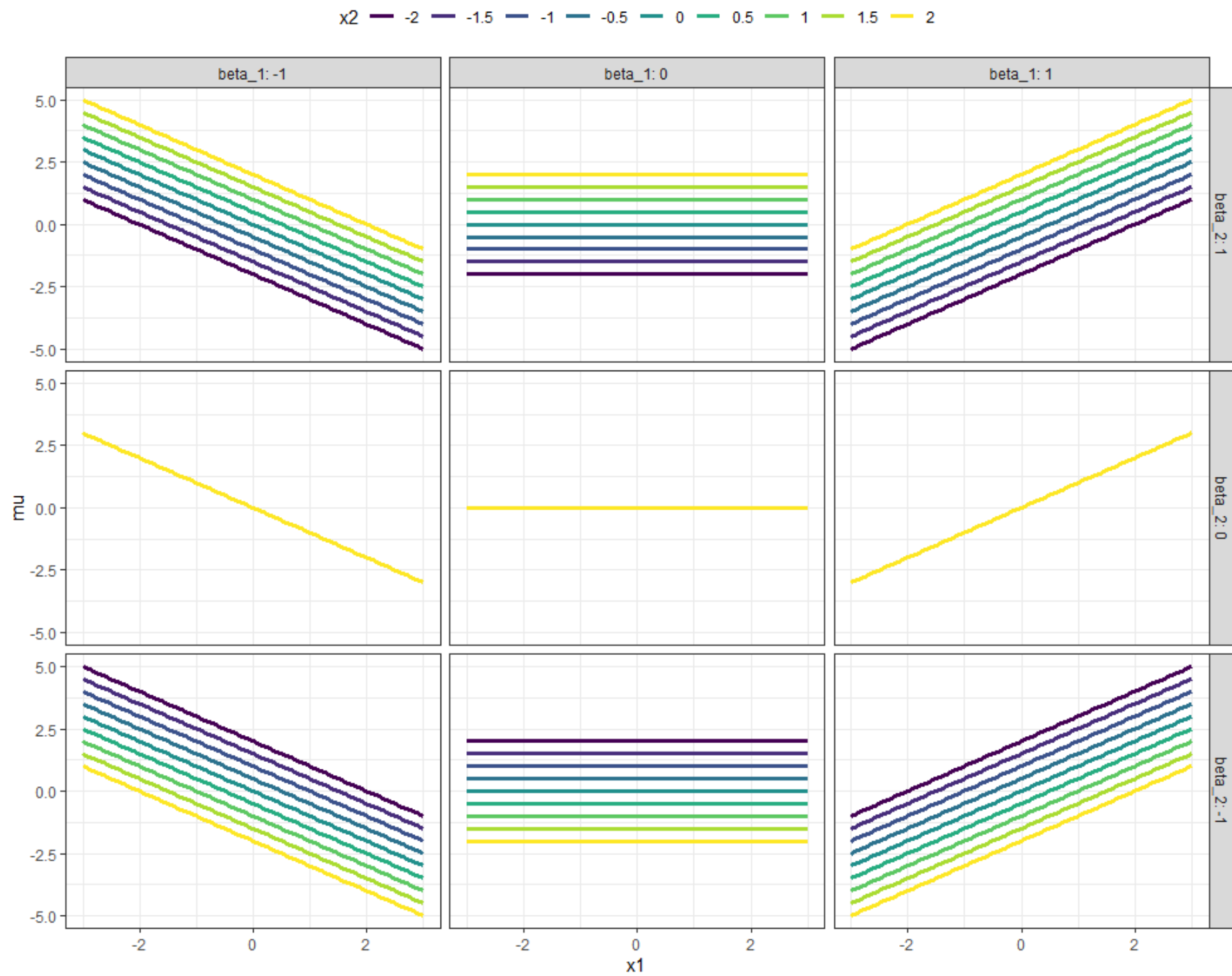
But...how does the second input influence the relationship associated with the first input?

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

- Or...let's consider an INTERPRETATION issue!
- How does x_2 impact the output behavior relative to x_1 ???

Visualize the trends for several different β_1 and β_2 combinations

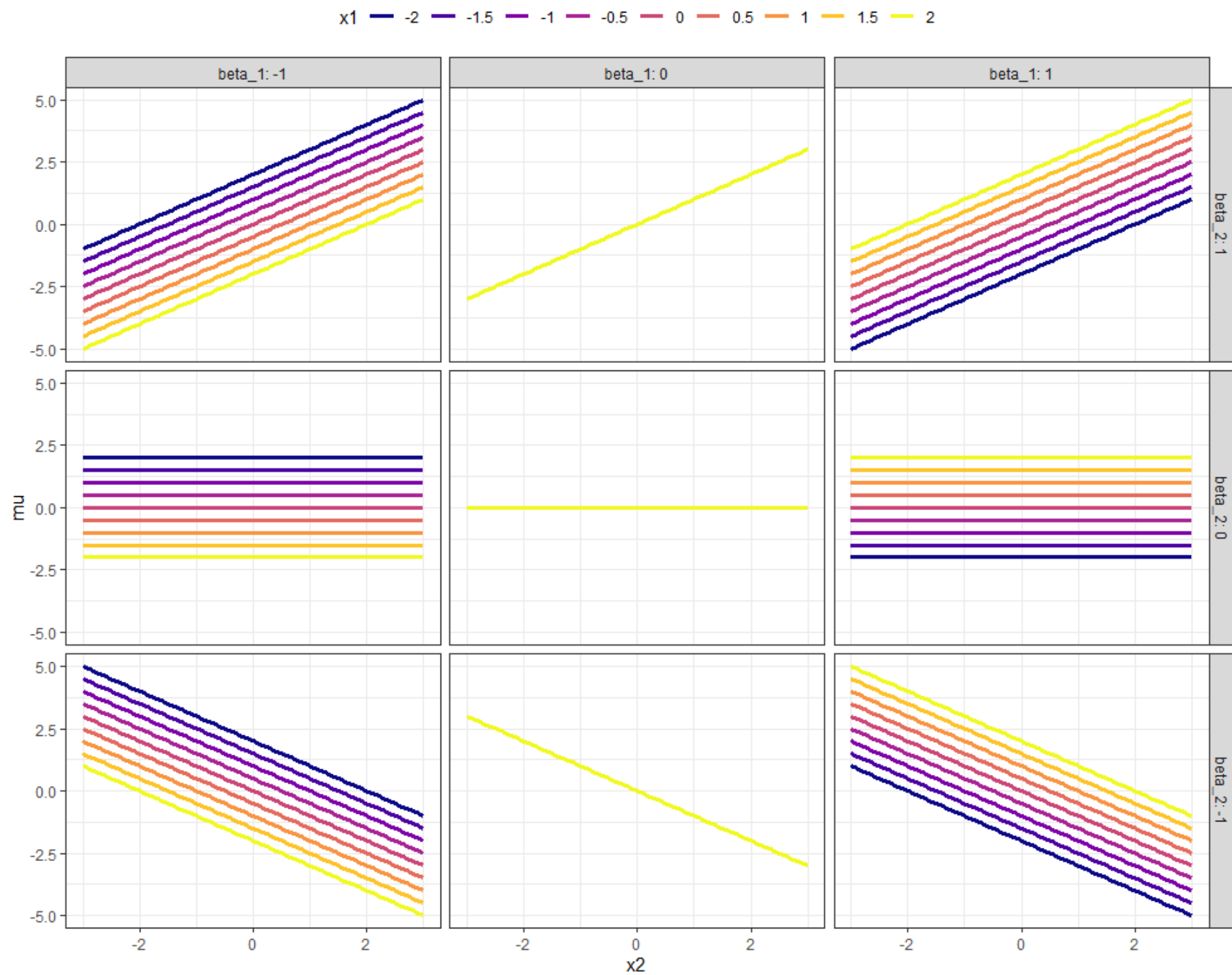
- Keep β_0 fixed at 0 for simplicity.
- Visualize the mean trend, μ , with respect to x_1 .
 - Colors denote values of x_2 .
 - Coefficients are displayed via facets.





Now ask the opposite question...how does x_1 impact the relationship with respect to x_2 ??

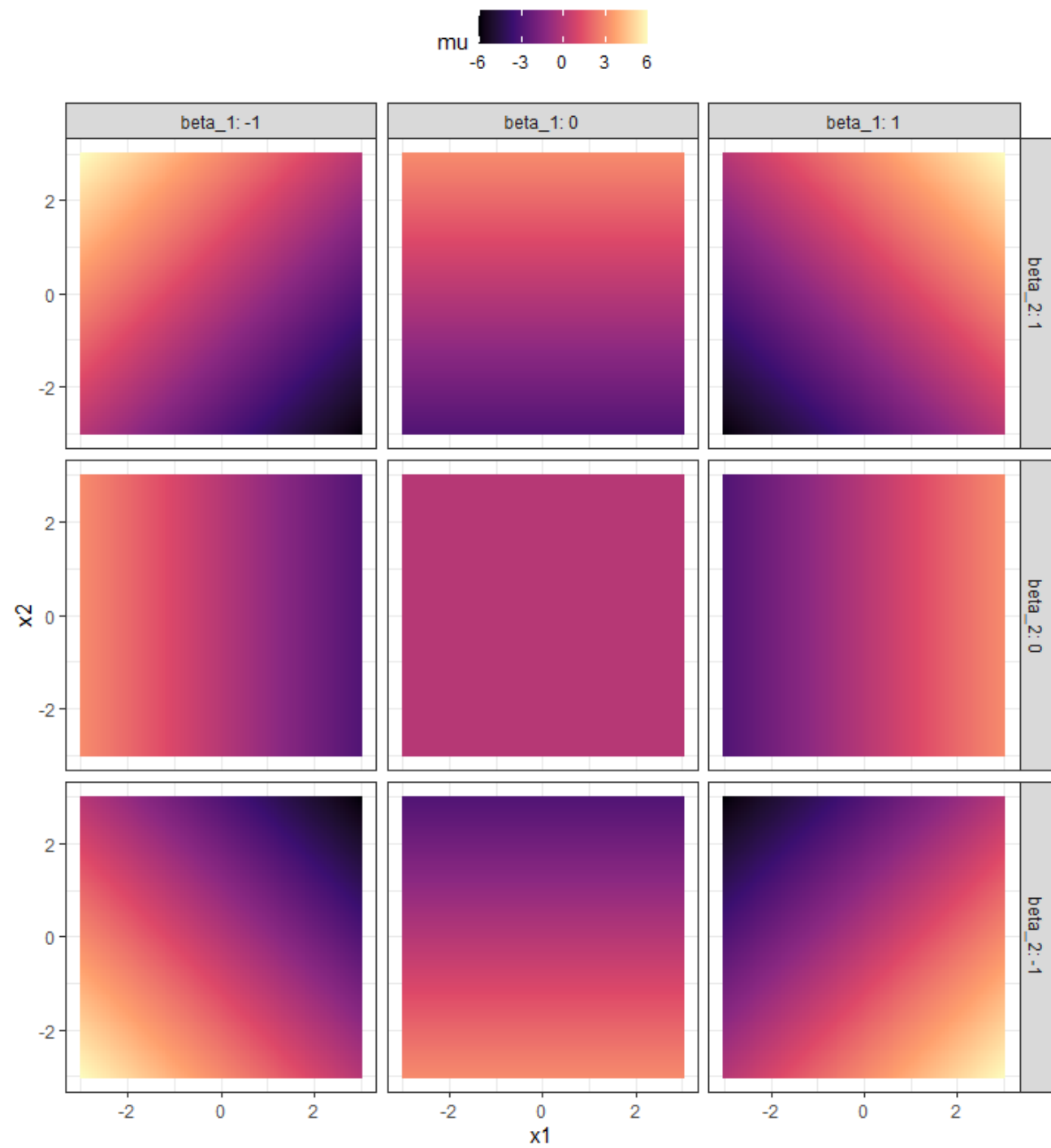
- Visualize the trends for several different β_1 and β_2 combinations
- Keep β_0 fixed at 0 for simplicity.
- Visualize the mean trend, μ , with respect to x_2 .
 - Colors denote values of x_1 .
 - Coefficients are displayed via facets.





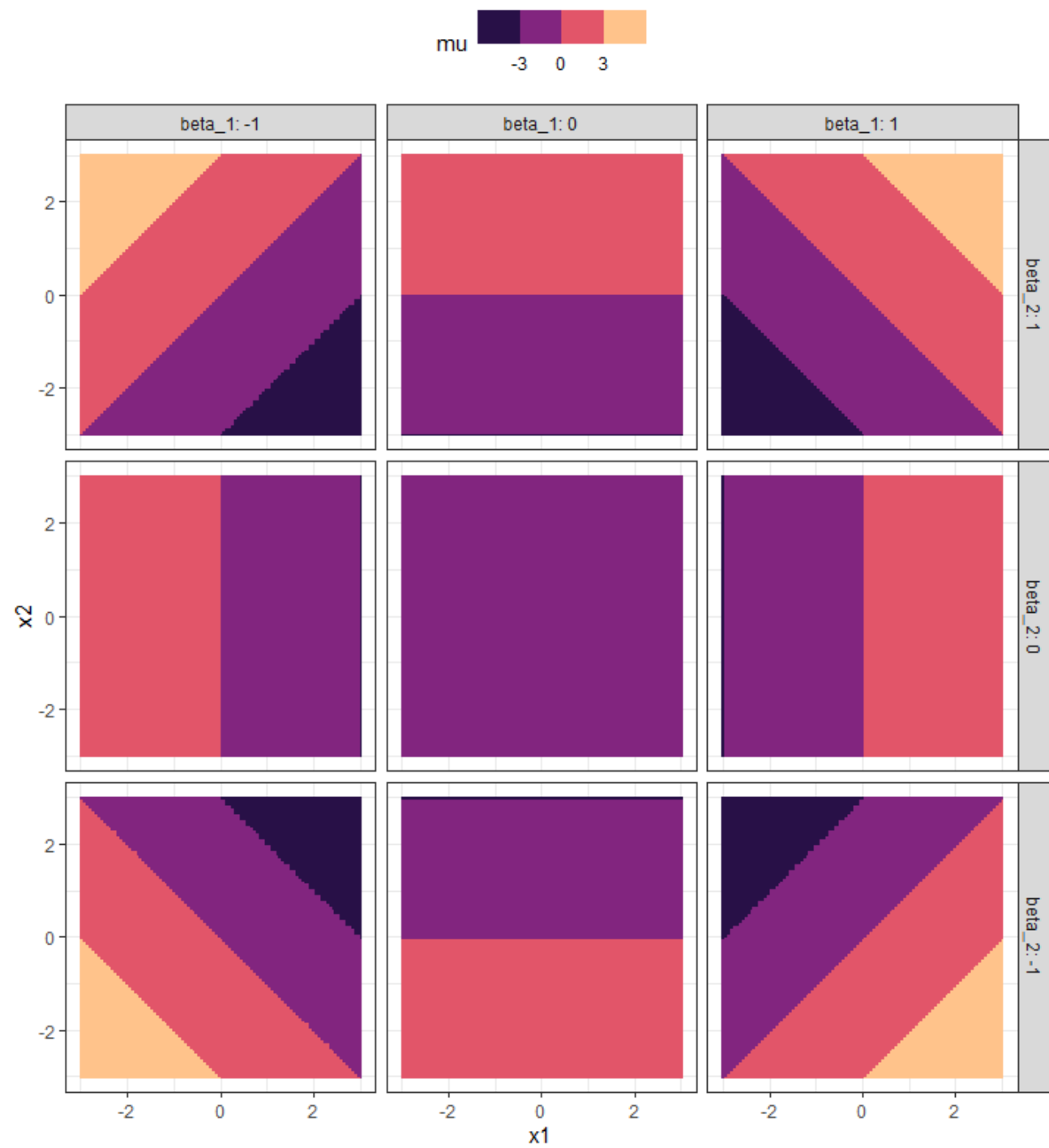
Visualize the trends for several different β_1 and β_2 combinations

- Keep β_0 fixed at 0 for simplicity.
- Visualize the mean trend, μ , as a surface with respect to x_1 and x_2 .
 - `geom_raster()` `fill` aesthetic denotes μ .
 - Coefficients are displayed via facets.



Sometimes, it is easier to visualize the behavior of a surface if the fill is “binned” or discretized

- More recent versions of `ggplot2` allow automatically binning the viridis fill palette with `scale_fill_viridis_b()`



Are we limited to linear RELATIONSHIPS
between the trend and TWO inputs??

Are we limited to linear RELATIONSHIPS between the trend and TWO inputs??

- NO!!!!!!
- We could derive polynomials from each input:

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1}^2 + \beta_4 x_{n,2}^2$$

Are we limited to linear RELATIONSHIPS between the trend and TWO inputs??

- NO!!!!!!
- We could derive other non-linear basis functions associated with each input, such as SPLINES:

$$\mu_n = \beta_0 + \sum_{j1=1}^{J1} \left(\beta_{j1} \phi_{j1}(x_{n,1}) \right) + \sum_{j2=0}^{J2} \left(\beta_{j2} \phi_{j2}(x_{n,2}) \right)$$

But...let's consider something DIFFERENT...

- Let's include an INTERACTION between the two inputs!!!
- An INTERACTION is the statistics way of saying MULTIPLICATION or PRODUCT.
- Start simple by including the INTERACTION with the LINEAR features:

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1} x_{n,2}$$

But...let's consider something DIFFERENT...

- Let's include an INTERACTION between the two inputs!!!
- An INTERACTION is the statistics way of saying MULTIPLICATION or PRODUCT.
- Start simple by including the INTERACTION with the LINEAR features:

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1} x_{n,2}$$

Is this a LINEAR MODEL???

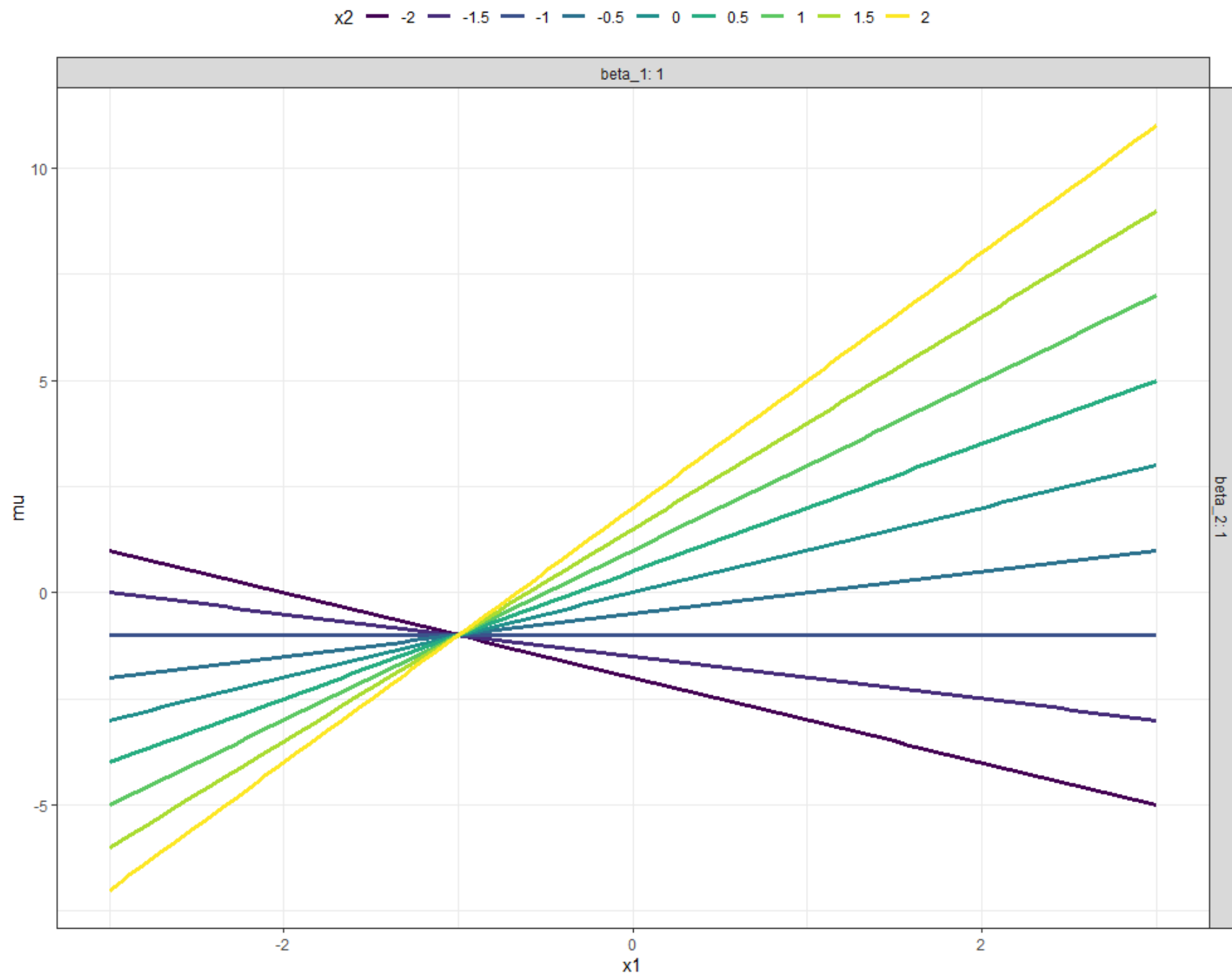
What if we have an interaction between the inputs?

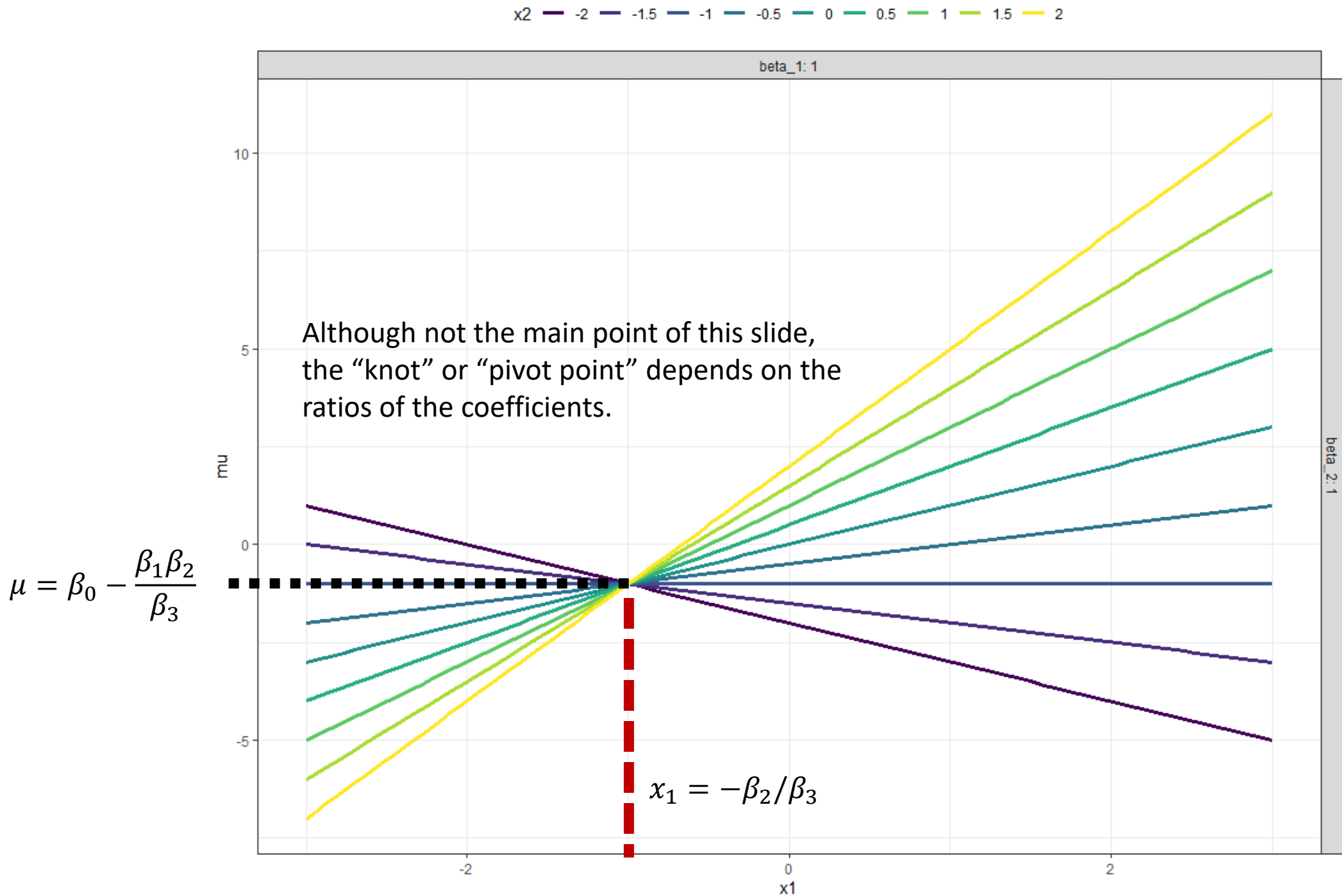
$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1} x_{n,2}$$

- Is this still a linear model?
- **YES!**
- The interaction “slope” β_3 is still linearly related to the mean trend.

How will the mean trend behave with respect to x_1 ?

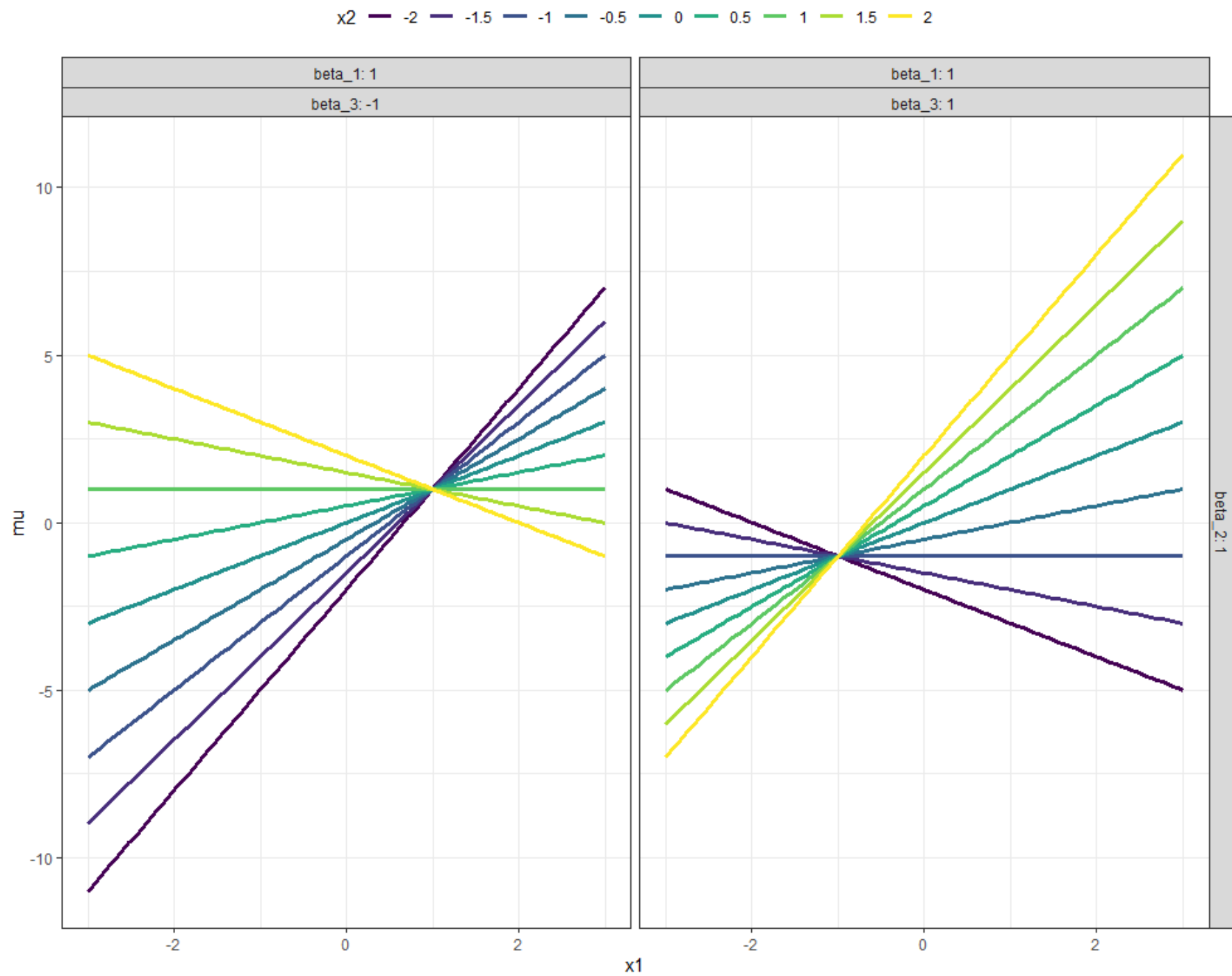
- If $\beta_0 = 0$, $\beta_1 = 1$, $\beta_2 = 1$, and $\beta_3 = 1$ how will μ depend on x_1 ?

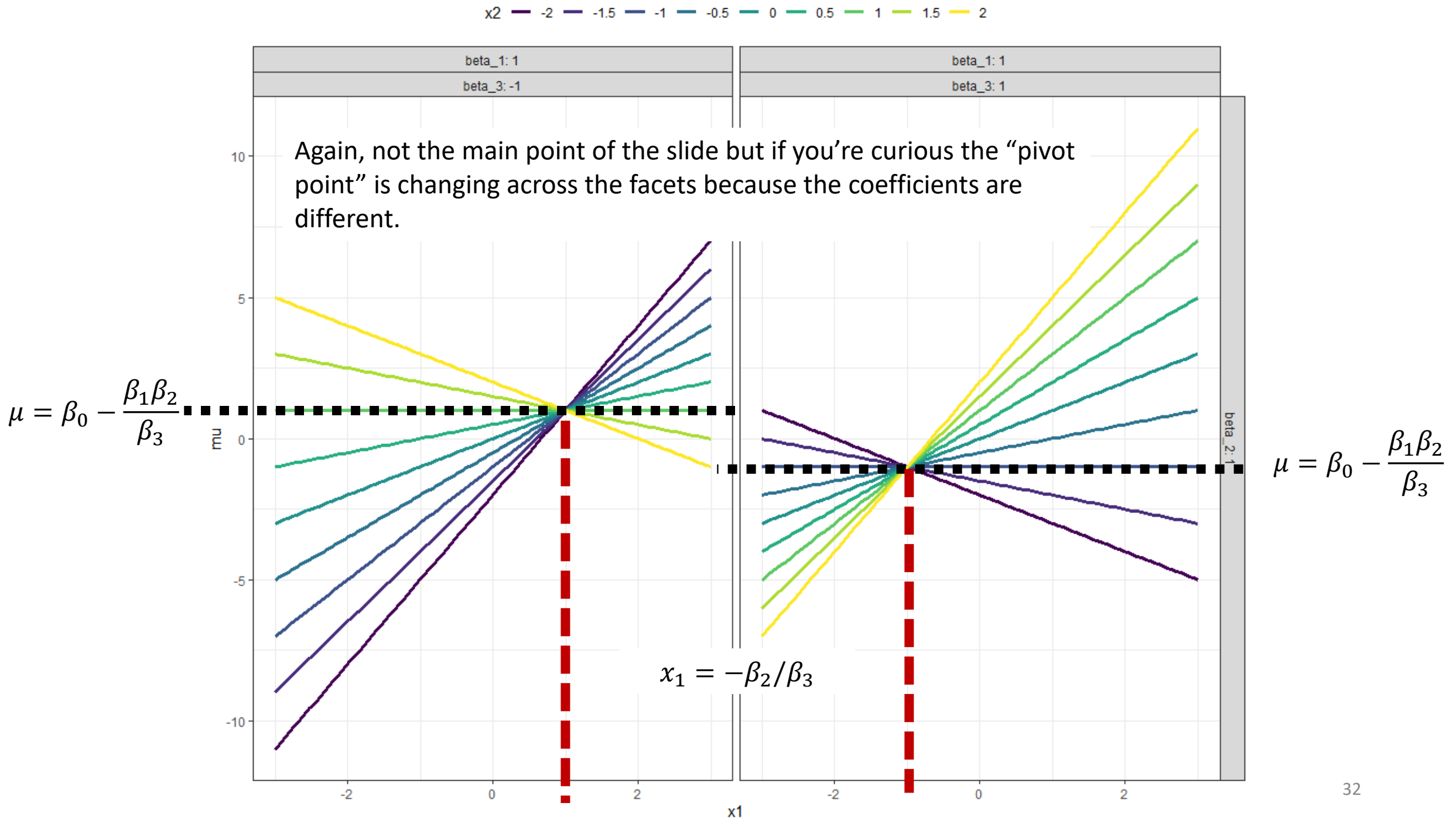




How will the mean trend behave with respect to x_1 ?

- If $\beta_0 = 0$, $\beta_1 = 1$, $\beta_2 = 1$, and $\beta_3 = -1$
how will μ depend on x_1 ?





The slope with respect x_1 depends on x_2 !

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1} x_{n,2}$$

- We can rewrite the expression with the interaction as:

$$\mu_n = \beta_0 + (\beta_1 + \beta_3 x_{n,2}) x_{n,1} + \beta_2 x_{n,2}$$

The slope with respect x_1 depends on x_2 !

$$\mu_n = \beta_0 + (\beta_1 + \beta_3 x_{n,2})x_{n,1} + \beta_2 x_{n,2}$$

- Define the effective slope: $\gamma_{n,1} = \beta_1 + \beta_3 x_{n,2}$

$$\mu_n = \beta_0 + \gamma_{n,1}x_{n,1} + \beta_2 x_{n,2}$$

The slope with respect x_1 depends on x_2 !

$$\mu_n = \beta_0 + (\beta_1 + \beta_3 x_{n,2})x_{n,1} + \beta_2 x_{n,2}$$

- Define the effective slope: $\gamma_{n,1} = \beta_1 + \beta_3 x_{n,2}$

$$\mu_n = \beta_0 + \gamma_{n,1}x_{n,1} + \beta_2 x_{n,2}$$

The **EFFECTIVE SLOPE** on x_1 depends on the MAIN EFFECT coefficient, β_1 , the INTERACTION coefficient, β_3 , **and the other INPUT x_2**

You can interpret the interaction as creating a model within a model!

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \gamma_{n,1}x_{n,1} + \beta_2x_{n,2}$$

$$\gamma_{n,1} = \beta_1 + \beta_3x_{n,2}$$

$$\beta_0, \beta_1, \beta_2, \beta_3 \mid \mu_\beta, \tau_\beta \sim \prod_{d=0}^{D=3} \text{normal}(\beta_d \mid \mu_\beta, \tau_\beta)$$

$$\sigma \mid \nu_\sigma \sim \text{Exp}(\sigma \mid \nu_\sigma)$$

You can interpret the interaction as creating a model within a model!

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

Equivalent to

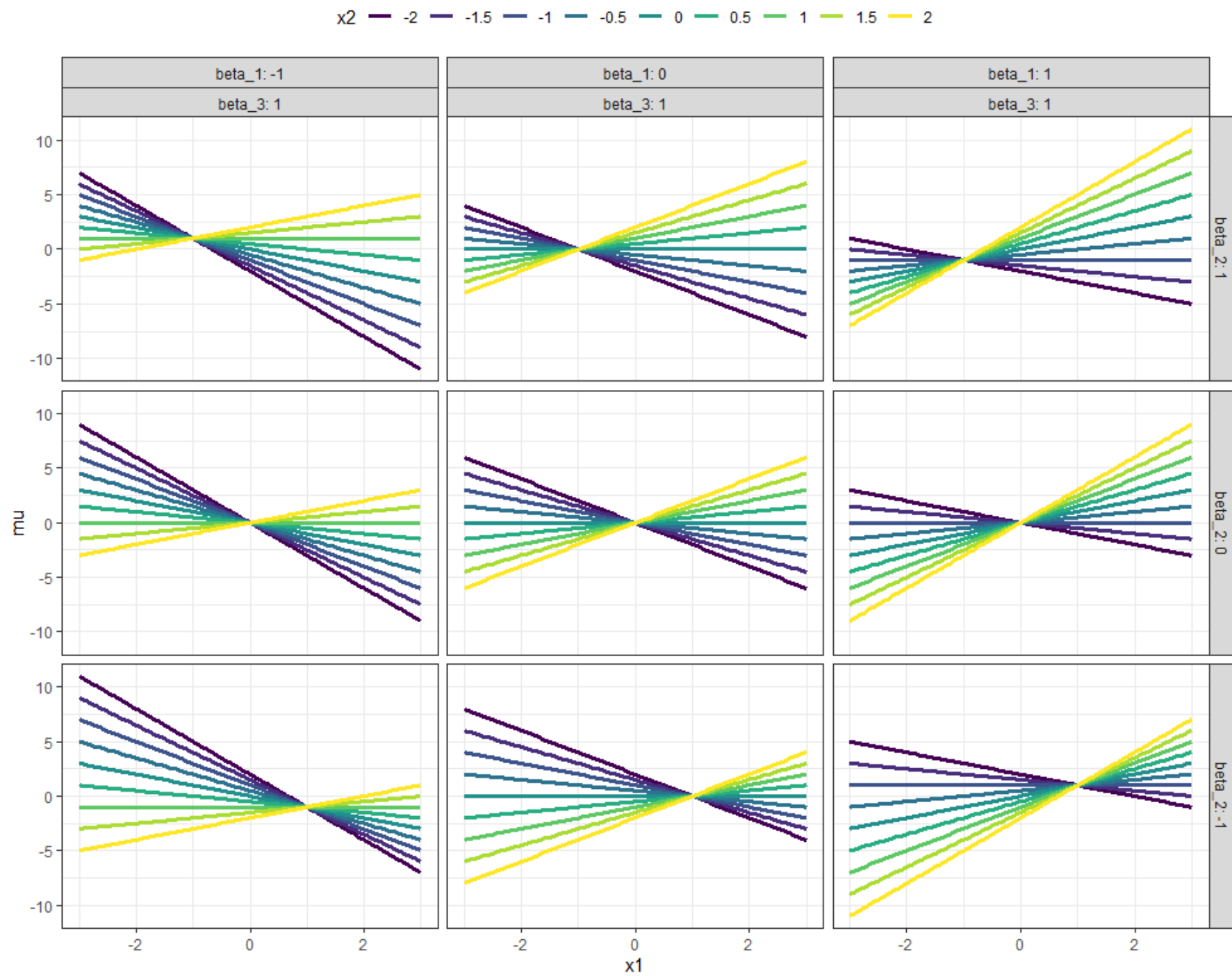
$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2} + \beta_3 x_{n,1} x_{n,2}$$

$$\beta_0, \beta_1, \beta_2, \beta_3 \mid \mu_\beta, \tau_\beta \sim \prod_{d=0}^{D=3} \text{normal}(\beta_d \mid \mu_\beta, \tau_\beta)$$

$$\sigma \mid \nu_\sigma \sim \text{Exp}(\sigma \mid \nu_\sigma)$$

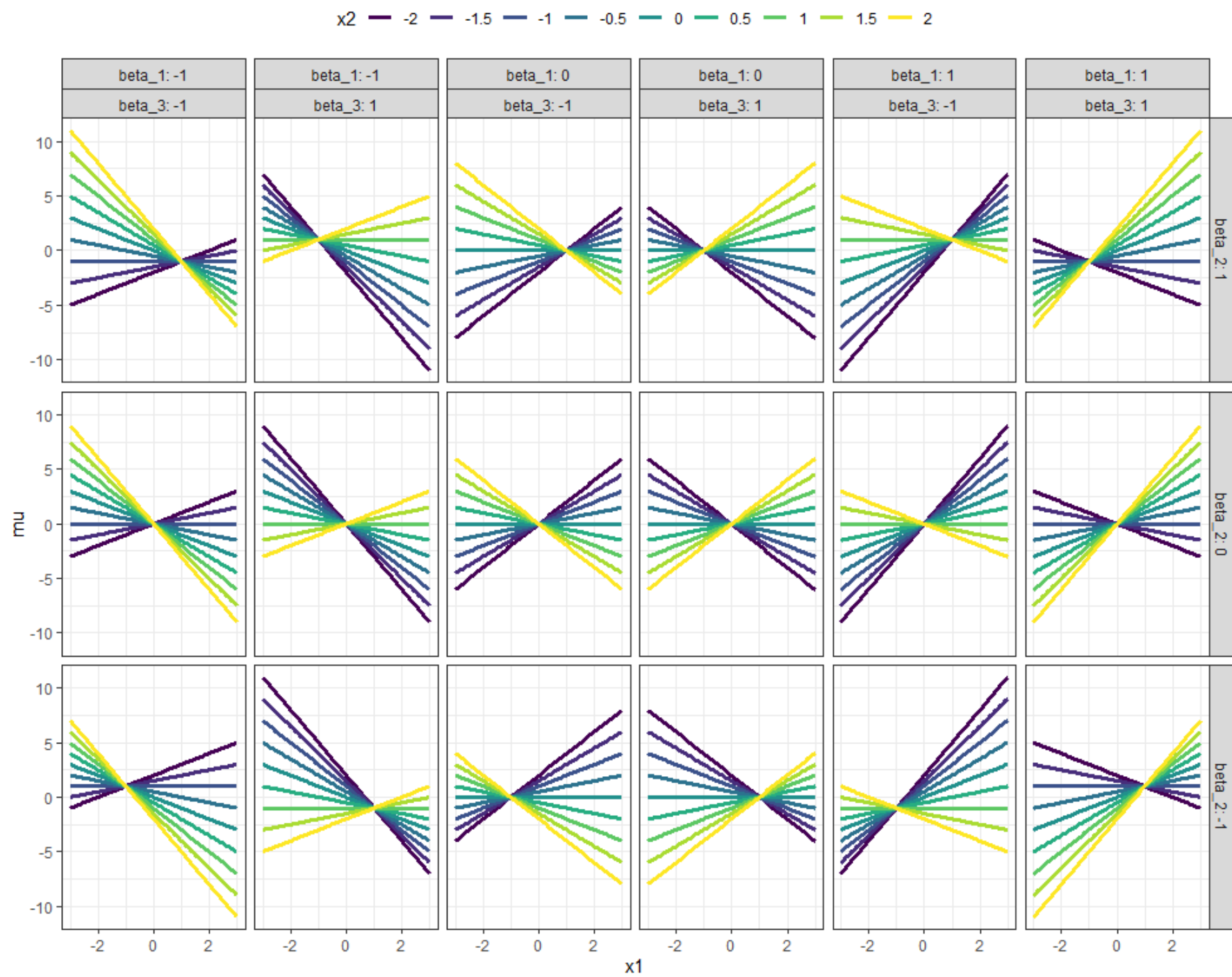
Visualize the trend with respect to x_1

- Use multiple values for the MAIN EFFECT coefficients β_1 and β_2
- Keep β_3 fixed at +1



Visualize the trend with respect to x_1

- Use multiple values for the MAIN EFFECT coefficients β_1 and β_2
- Use two values for β_3 at -1 and +1



But...what if we had instead defined the effective slope on x_2 ...

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + (\beta_2 + \beta_3 x_{n,1}) x_{n,2}$$

- Define the effective slope: $\gamma_{n,2} = \beta_2 + \beta_3 x_{n,1}$

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \gamma_{n,2} x_{n,2}$$

Is this valid?

But...what if we had instead defined the effective slope on x_2 ...

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + (\beta_2 + \beta_3 x_{n,1}) x_{n,2}$$

- Define the effective slope: $\gamma_{n,2} = \beta_2 + \beta_3 x_{n,1}$

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \gamma_{n,2} x_{n,2}$$

YES!

You can interpret the interaction as creating a model within a model...but we can interpret the behavior multiple ways! Be careful!

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \gamma_{n,1}x_{n,1} + \beta_2x_{n,2}$$

$$\gamma_{n,1} = \beta_1 + \beta_3x_{n,2}$$

OR

$$\mu_n = \beta_0 + \beta_1x_{n,1} + \gamma_{n,2}x_{n,2}$$

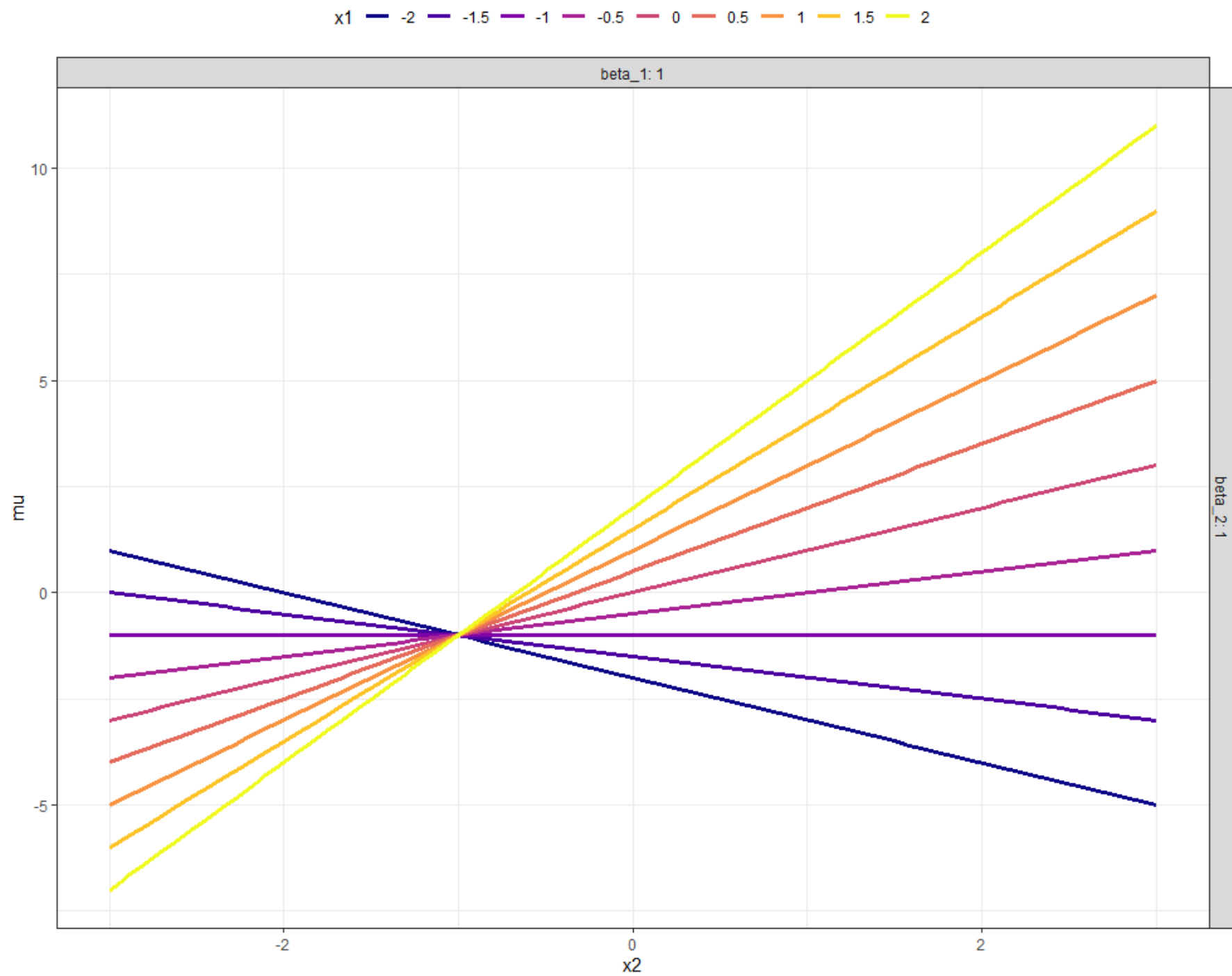
$$\gamma_{n,2} = \beta_2 + \beta_1x_{n,1}$$

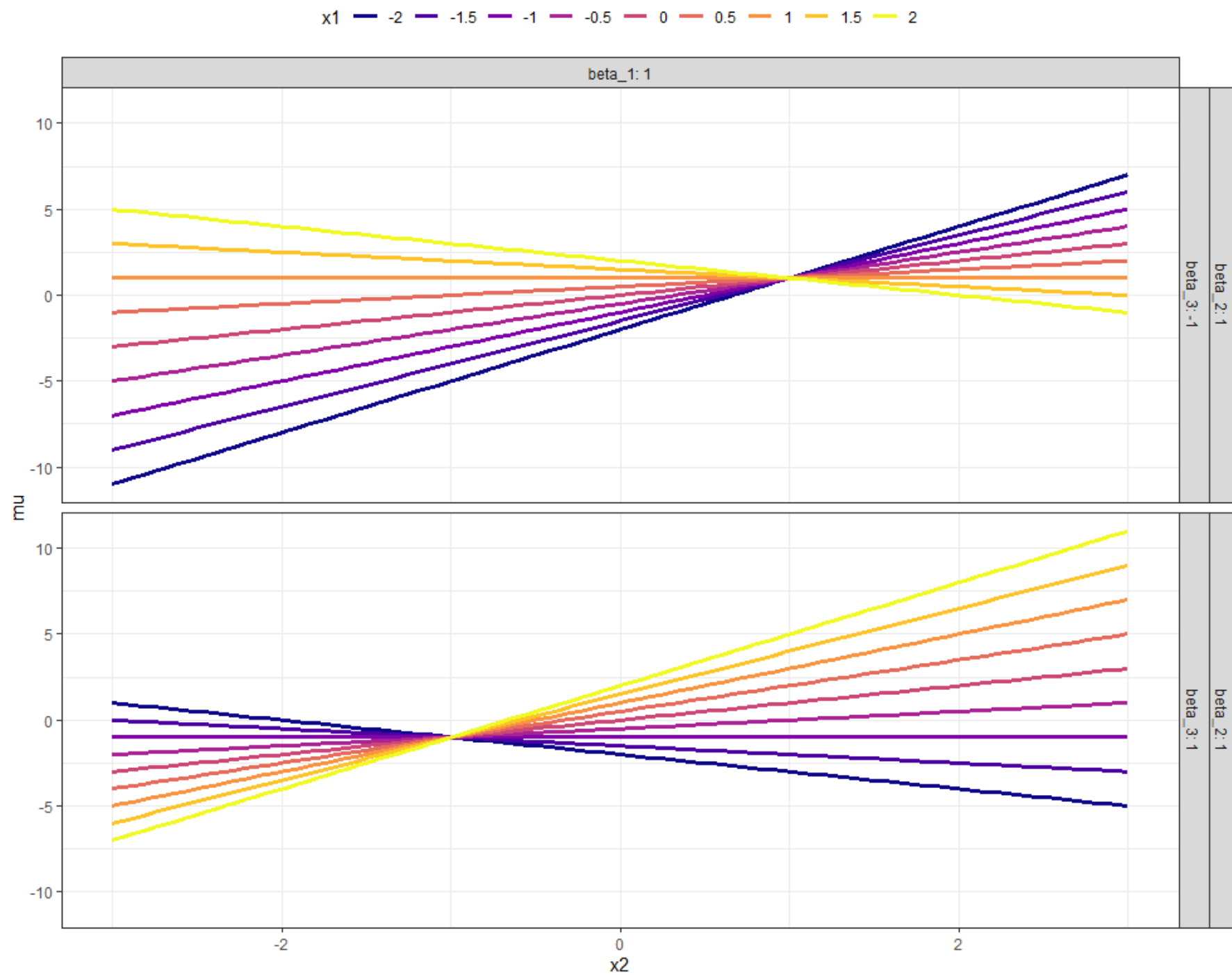
$$\beta_0, \beta_1, \beta_2, \beta_3 \mid \mu_\beta, \tau_\beta \sim \prod_{d=0}^{D=3} \text{normal}(\beta_d \mid \mu_\beta, \tau_\beta)$$

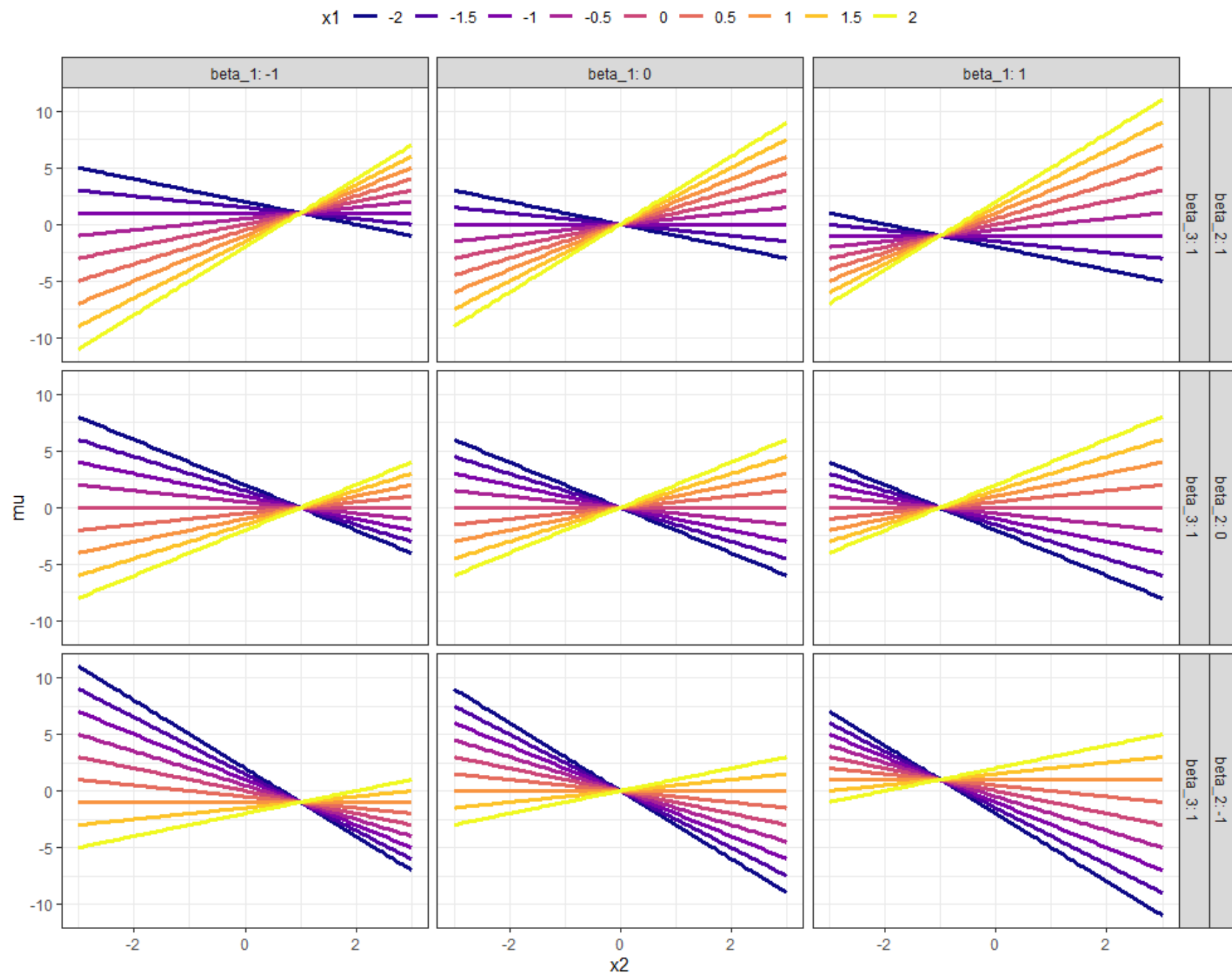
$$\sigma \mid \nu_\sigma \sim \text{Exp}(\sigma \mid \nu_\sigma)$$

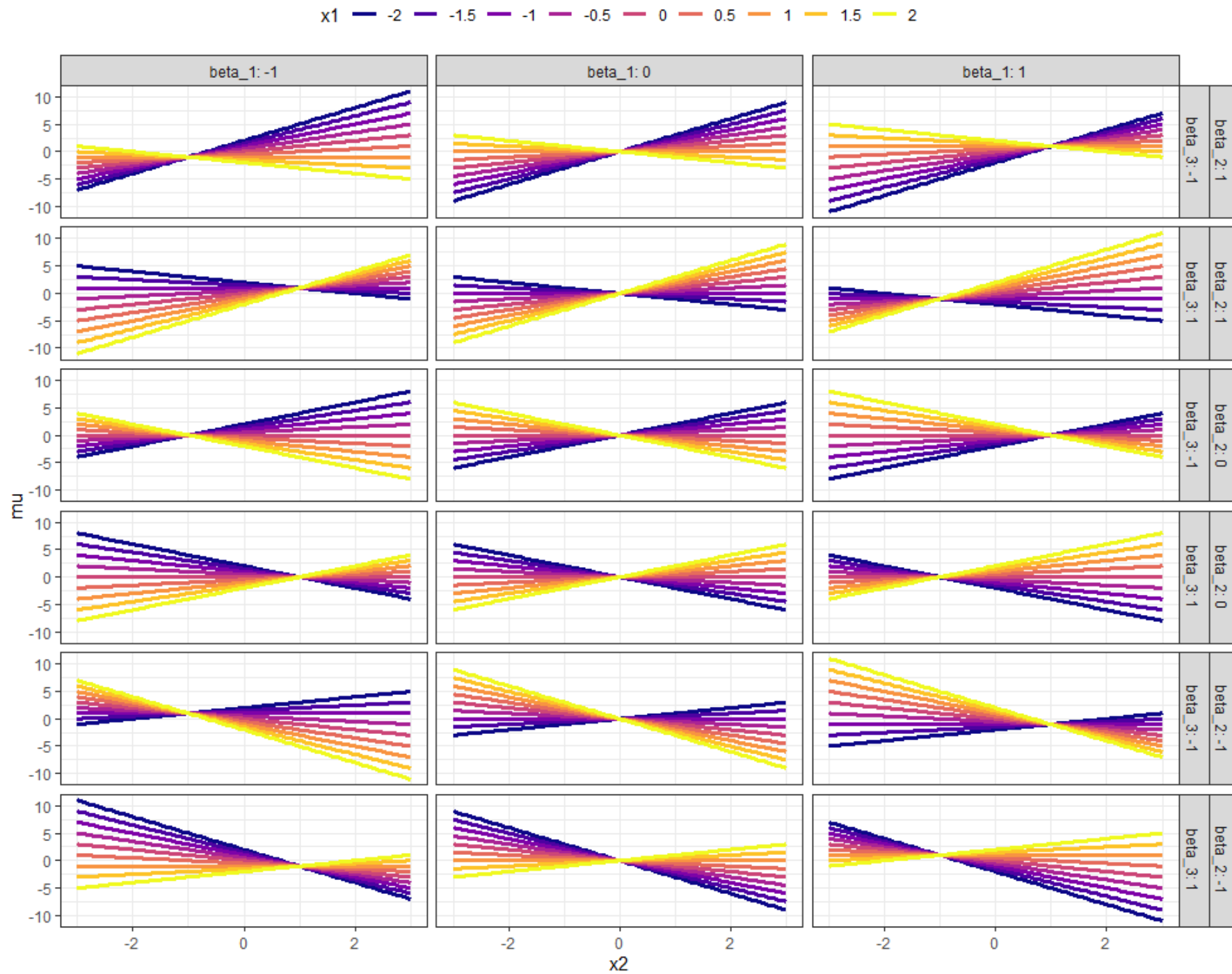
Can repeat each of the visualizations from before

- Study the mean trend with respect to x_2 .
- Use facets to study the impact of various INTERACTION and MAIN EFFECT coefficient values.



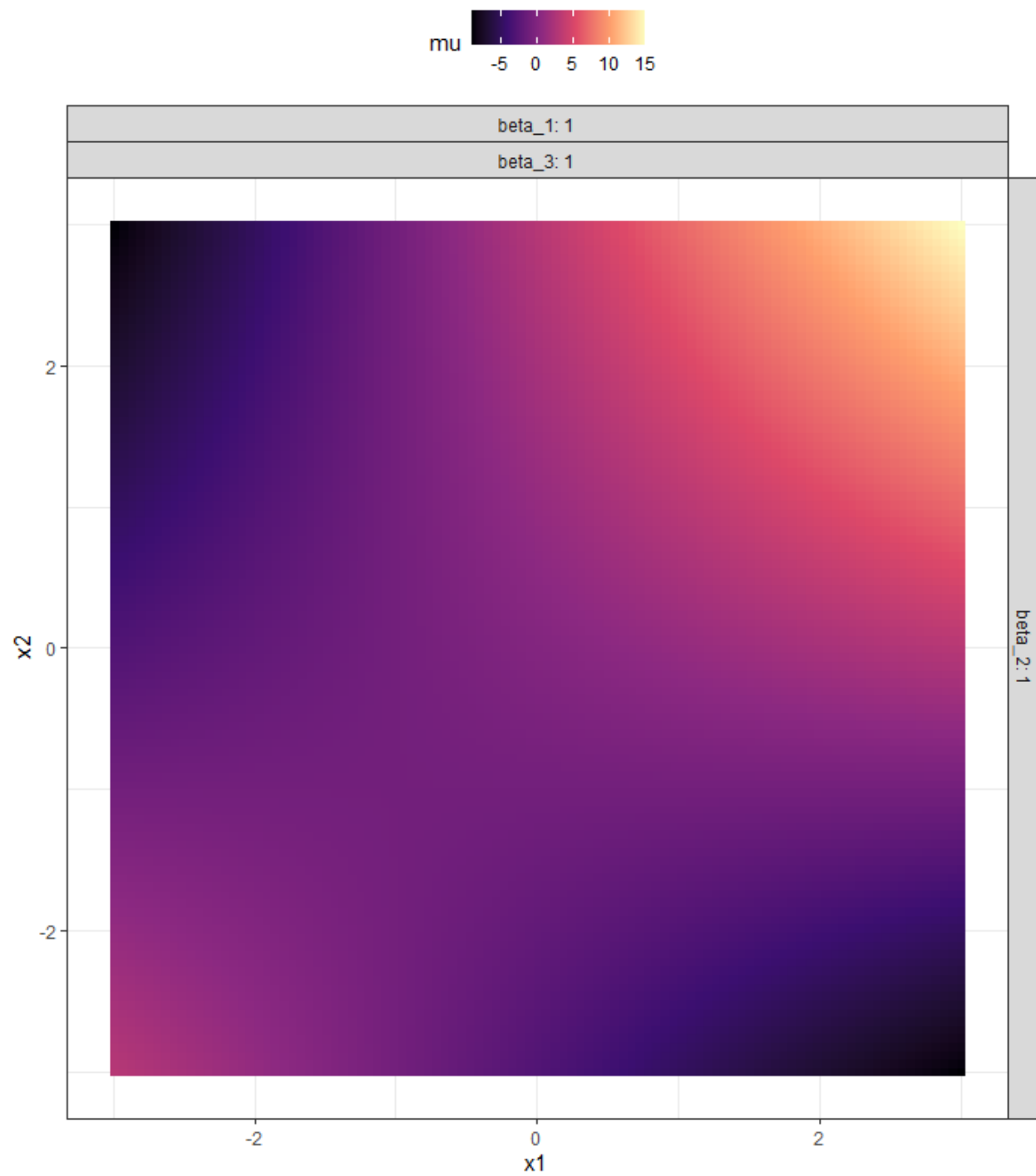




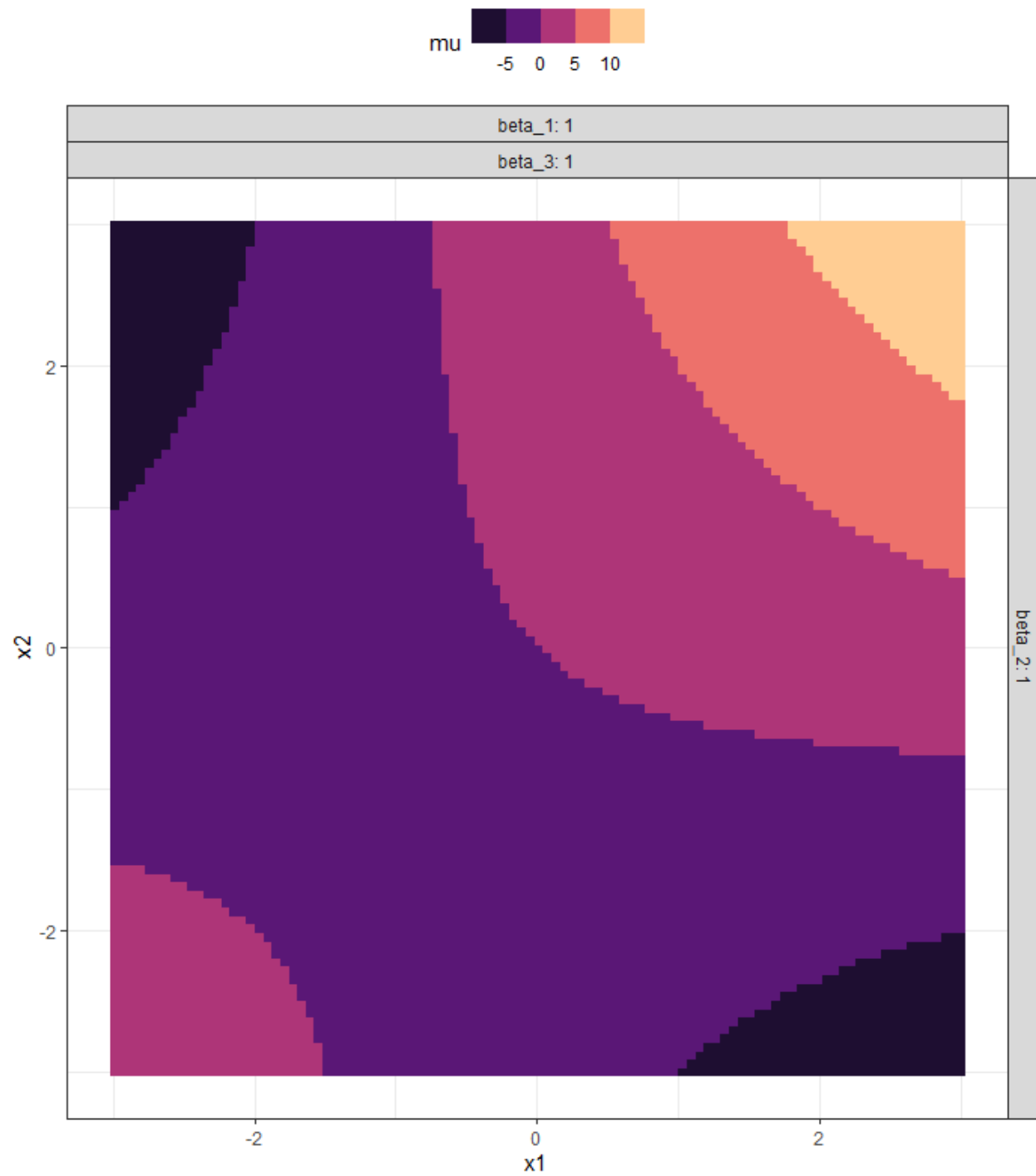


What does the surface of the mean, μ , look like when we have an interaction term?

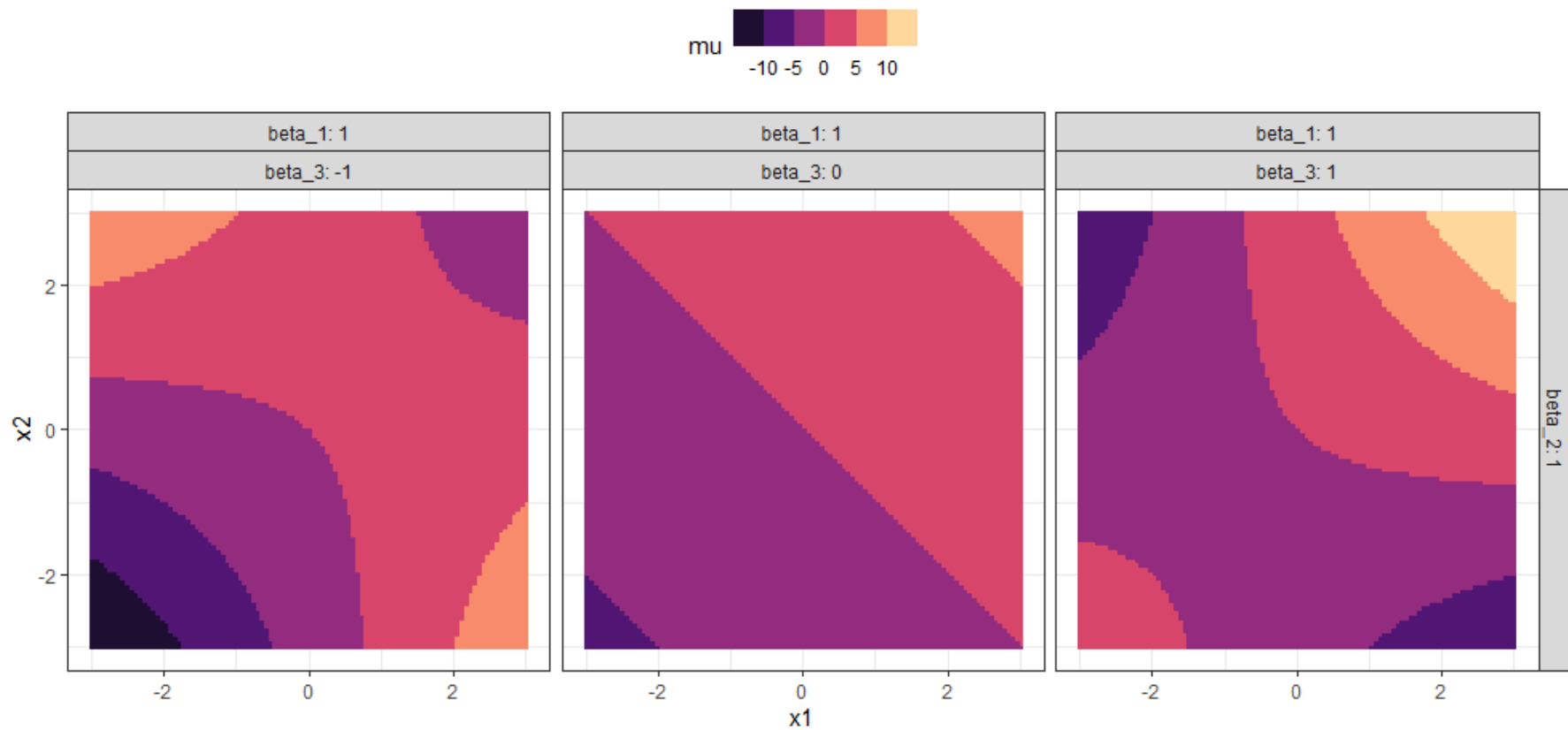
- Consider $\beta_0 = 0$, $\beta_1 = 1$, $\beta_2 = 1$, and $\beta_3 = 1$



Bin or discretize the fill to make it easier to visualize the behavior of the surface



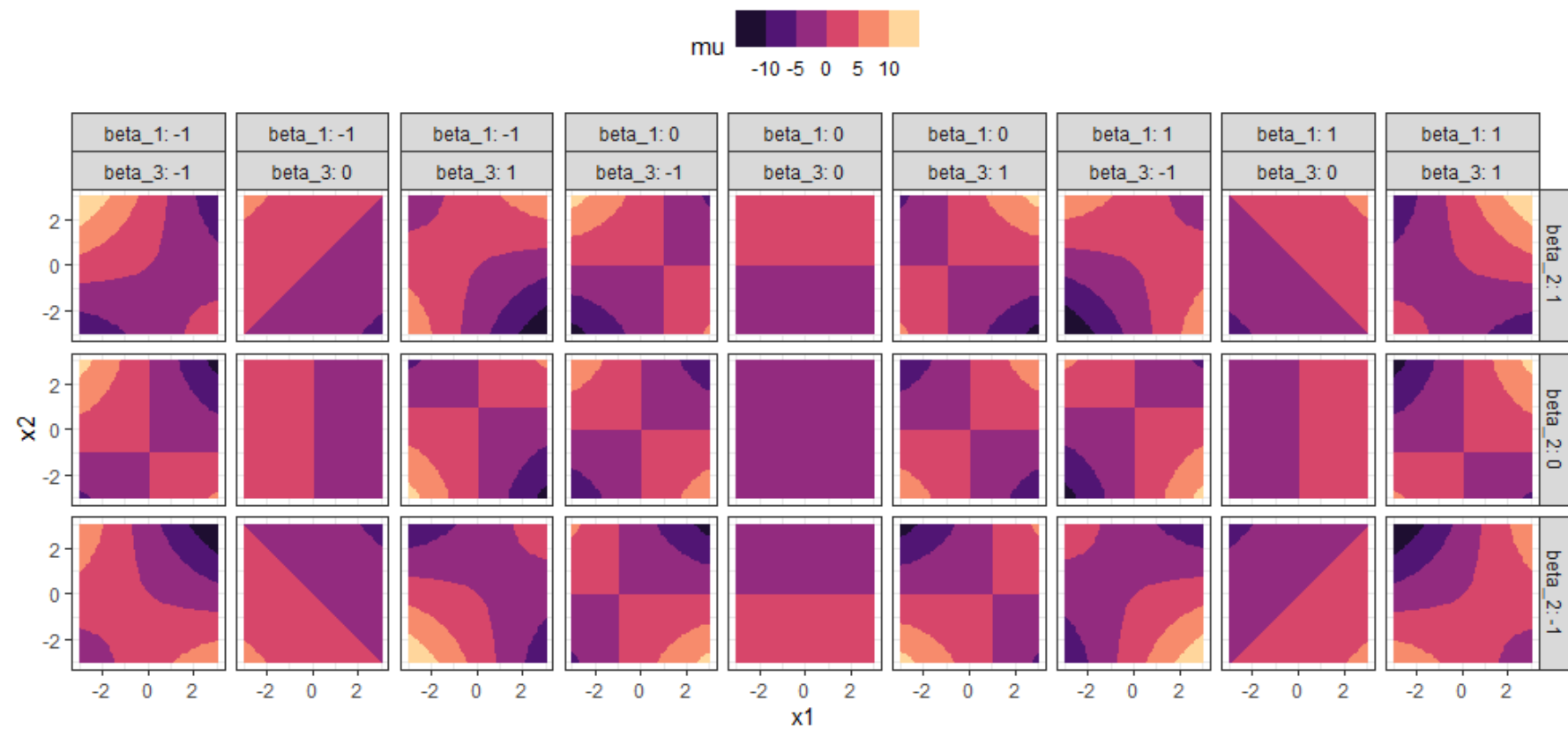
Consider β_3 equal to -1, 0, and +1



$\beta_3 = 0$ is just the
additive model!

Vary the MAIN EFFECT and INTERACTION coefficients

- Visualize the surface of μ with respect to x_1 and x_2
- Facets denote the combinations of the coefficients:
 - $\beta_1 = -1, 0, +1$
 - $\beta_2 = -1, 0, +1$
 - $\beta_3 = -1, 0, +1$



When we have multiple inputs...

- There's something else we **MUST** be worried about!!!

What happens if the inputs, x_1 and x_2 , are related to each other?

- In other words, **what if the two inputs are highly correlated?**
- For example, let's assume we can define a linear relationship between the two:

$$x_2 = c + dx_1$$

Let's consider a linear additive relationship (no interaction) for simplicity...

- The mean trend is:

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

- Since x_2 is related to x_1 we can rewrite the mean trend as:

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 (c + d x_{n,1})$$

Rearrange the expression for the mean trend

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 (c + dx_{n,1})$$



Distribute β_2 through parentheses

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 c + \beta_2 dx_{n,1}$$

Rearrange the expression for the mean trend

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 (c + dx_{n,1})$$



Distribute β_2 through parentheses

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 c + \beta_2 dx_{n,1}$$



Rearrange

$$\mu_n = \beta_0 + \beta_2 c + \beta_1 x_{n,1} + \beta_2 dx_{n,1}$$

Rearrange the expression for the mean trend

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 (c + dx_{n,1})$$



Distribute β_2 through parentheses

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 c + \beta_2 dx_{n,1}$$



Rearrange

$$\mu_n = \beta_0 + \beta_2 c + \beta_1 x_{n,1} + \beta_2 dx_{n,1}$$



Group the constants and terms involving x_1

$$\mu_n = (\beta_0 + \beta_2 c) + (\beta_1 + \beta_2 d)x_{n,1}$$

Rearrange the expression for the mean trend

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 (c + d x_{n,1})$$



Distribute β_2 through parentheses

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 c + \beta_2 d x_{n,1}$$



Rearrange

$$\mu_n = \beta_0 + \beta_2 c + \beta_1 x_{n,1} + \beta_2 d x_{n,1}$$



Group the constants and terms involving x_1

$$\mu_n = (\beta_0 + \beta_2 c) + (\beta_1 + \beta_2 d) x_{n,1}$$



EFFECTIVE INTERCEPT



EFFECTIVE SLOPE on x_1

$$\mu_n = \zeta_0 + \zeta_1 x_{n,1}$$

Linearly related inputs are problematic because the effective model has fewer unknowns!

- The effective model has just 2 unknowns, ζ_0 and ζ_1 !

$$\mu_n = \zeta_0 + \zeta_1 x_{n,1}$$

- But we are trying to learn 3 coefficients!

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

There are many combinations of β_1 and β_2 that can give the same value of ζ_1

- The effective slope on x_1 is: $\zeta_1 = \beta_1 + \beta_2 d$
- For simplicity, assume $d = 1$, then $\zeta_1 = \beta_1 + \beta_2$.
- Increasing β_1 by 2 units can be balanced out by decreasing β_2 by 2 units!
- The effective slope, ζ_1 , will be the same!

There are many combinations of β_1 and β_2 that can give the same value of ζ_1

- The effective slope on x_1 is: $\zeta_1 = \beta_1 + \beta_2 d$
- For simplicity, assume $d = 1$, then $\zeta_1 = \beta_1 + \beta_2$.
- Many combinations of β_1 and β_2 yield the same ζ_1 .
- We will be certain a-posterior about ζ_1 but **UNCERTAIN** about β_1 and β_2 !!

This “balance” or trade-off is represented by the covariance between the parameters!

- When σ is fixed or known, and we have an infinitely diffuse prior on the β coefficients, the posterior covariance matrix is:

$$\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$$

- The matrix sum of squares, $\mathbf{X}^T \mathbf{X}$, represents the relationship between the inputs.

Simulation example for 2 inputs that are additively related to the mean trend

$$\mu_n = \beta_0 + \beta_1 x_{n,1} + \beta_2 x_{n,2}$$

- For simplicity, assume that the 2 inputs have marginal Gaussian distributions with mean 0 and standard deviation 1 (marginal standard normals).

Generate 100 random input combinations assuming 3 different correlation coefficients between the inputs, ρ .

- The inputs therefore have a MVN distribution:

$$x_1, x_2 \mid \boldsymbol{\mu}, \boldsymbol{\Sigma} \sim \mathcal{N} \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \mid \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} \right)$$

- Generate 100 random values of x_1 and x_2 when:
 - $\rho = -0.9$
 - $\rho = 0$
 - $\rho = 0.9$

Code to generate 100 random observations when $\rho = 0$

```
### for simplicity assume that the two inputs are distributed with MVN
### with marginal standard deviations of 1 and means 0

num_inputs <- 2

### start with correlation coefficient of 0

x_mean <- rep(0, num_inputs)

x_covmat_rho_zero <- diag(num_inputs)

x_covmat_rho_zero

### generate 100 random observations of the two inputs

set.seed(7131)

xinputs_rho_zero <- MASS::mvrnorm(n = 100, mu=x_mean, Sigma = x_covmat_rho_zero) %>%
  as.data.frame() %>% tibble::as_tibble() %>%
  purrr::set_names(sprintf("x%d", 1:num_inputs))
```

Change the covariance matrix to generate the random values for the other two cases

```
### now generate 100 random samples when the input correlation
### is very high, 0.9

x_covmat_rho_high <- matrix(c(1, 0.9,
                             0.9, 1),
                           nrow = 2,
                           byrow = TRUE)

x_covmat_rho_high

set.seed(7131)

xinputs_rho_high <- MASS::mvrnorm(n = 100, mu=x_mean, Sigma = x_covmat_rho_high) %>%
  as.data.frame() %>% tibble::as_tibble() %>%
  purrr::set_names(sprintf("x%d", 1:num_inputs))
```

```
### what about if the inputs are "highly negatively" correlated or
### anticorrelated?

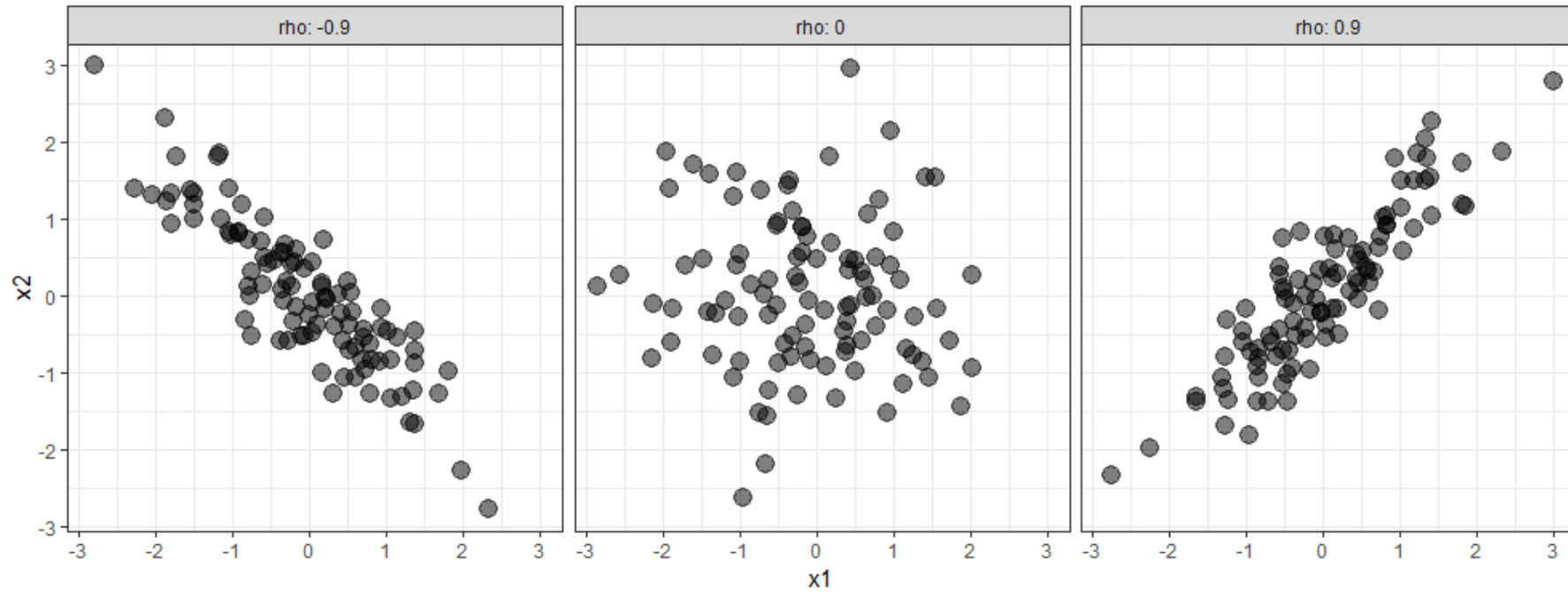
x_covmat_rho_neg <- matrix(c(1, -0.9,
                             -0.9, 1),
                           nrow = 2,
                           byrow = TRUE)

x_covmat_rho_neg

set.seed(7131)

xinputs_rho_neg <- MASS::mvrnorm(n = 100, mu=x_mean, Sigma = x_covmat_rho_neg) %>%
  as.data.frame() %>% tibble::as_tibble() %>%
  purrr::set_names(sprintf("x%d", 1:num_inputs))
```

Randomly generated values of the 2 inputs x_1 and x_2
for 3 different values of the correlation coefficient, ρ



Calculate the posterior covariance matrix

- Assume infinitely diffuse priors on the β parameters for simplicity.
- Assume that the noise (residual error), σ , is known and is equal to 1 for simplicity.
- Remember, we are focusing on the relationship between the parameters due to the relationship between the inputs.

Calculate the design matrix, then the sum of squares matrix, and lastly the posterior covariance matrix first for the $\rho = 0$ case

```
> ### create the design matrix
>
> X_rho_zero <- model.matrix(~ x1 + x2, xinputs_rho_zero)
```

Design matrix, $\mathbf{X}$

```
> X_rho_zero %>% head()
      (Intercept)      x1      x2
1             1 -1.4166132 -0.2061031
2             1 -0.1110282  0.7821542
3             1 -0.7421027 -1.5156365
4             1  2.0162510 -0.9341914
5             1  0.7780526  0.5072637
6             1  1.5716407 -0.1517222
```

```
> ### matrix sum of squares
> SSmat_rho_zero <- t(X_rho_zero) %*% X_rho_zero
```

Matrix sum of squares, $\mathbf{X}^T \mathbf{X}$

```
> SSmat_rho_zero
      (Intercept)      x1      x2
(Intercept) 100.000000 -11.731428  4.405114
x1          -11.731428 113.460143 -8.796596
x2           4.405114  -8.796596 97.088302
```

```
> ### calculate the posterior covariance matrix assuming
> ### the noise (residual error), sigma, is 1
```

Posterior covariance matrix, $\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$ assuming $\sigma = 1$

```
> solve(SSmat_rho_zero)
      (Intercept)      x1      x2
(Intercept) 0.0101357931 0.001019516 -0.0003675113
x1          0.0010195164 0.008978567  0.0007672370
x2          -0.0003675113 0.000767237  0.0103860916
```

```
> ### look at the posterior correlation between the 2 slopes
> ### it's very small!
```

Posterior correlation matrix, the correlation between β_1 and β_2 is the [2, 3] (or [3, 2]) element in the matrix

```
> cov2cor(solve(SSmat_rho_zero))
      (Intercept)      x1      x2
(Intercept) 1.00000000 0.10687149 -0.03581919
x1          0.10687149 1.00000000  0.07945111
x2          -0.03581919 0.07945111  1.00000000
```

Calculate the design matrix, then the sum of squares matrix, and lastly the posterior covariance matrix first for the $\rho = 0$ case

```
> ### create the design matrix
>
> X_rho_zero <- model.matrix(~ x1 + x2, xinputs_rho_zero)
```

Design matrix, $\mathbf{X}$

```
> X_rho_zero %>% head()
      (Intercept)      x1      x2
1             1 -1.4166132 -0.2061031
2             1 -0.1110282  0.7821542
3             1 -0.7421027 -1.5156365
4             1  2.0162510 -0.9341914
5             1  0.7780526  0.5072637
6             1  1.5716407 -0.1517222
```

```
> ### matrix sum of squares
> SSmat_rho_zero <- t(X_rho_zero) %*% X_rho_zero
>
> SSmat_rho_zero
```

```
      (Intercept)      x1      x2
(Intercept) 100.000000 -11.731428  4.405114
x1          -11.731428 113.460143 -8.796596
x2           4.405114  -8.796596 97.088302
```

```
> ### calculate the posterior covariance matrix assuming
> ### the noise (residual error), sigma, is 1
```

```
> solve(SSmat_rho_zero)
      (Intercept)      x1      x2
(Intercept) 0.0101357931 0.001019516 -0.0003675113
x1          0.0010195164 0.008978567  0.0007672370
x2          -0.0003675113 0.000767237  0.0103860916
```

```
> ### look at the posterior correlation between the 2 slopes
> ### it's very small!
```

```
> cov2cor(solve(SSmat_rho_zero))
      (Intercept)      x1      x2
(Intercept) 1.00000000 0.10687149 0.03581919
x1          0.10687149 1.00000000 0.07945111
x2          -0.03581919 0.07945111 1.00000000
```

Matrix sum of squares, $\mathbf{X}^T \mathbf{X}$

Posterior covariance matrix, $\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$ assuming $\sigma = 1$

Posterior correlation matrix, the correlation between β_1 and β_2 is the [2, 3] (or [3, 2]) element in the matrix

Posterior correlation is CLOSE to ZERO!

Same set of actions for the other two correlation coefficient cases

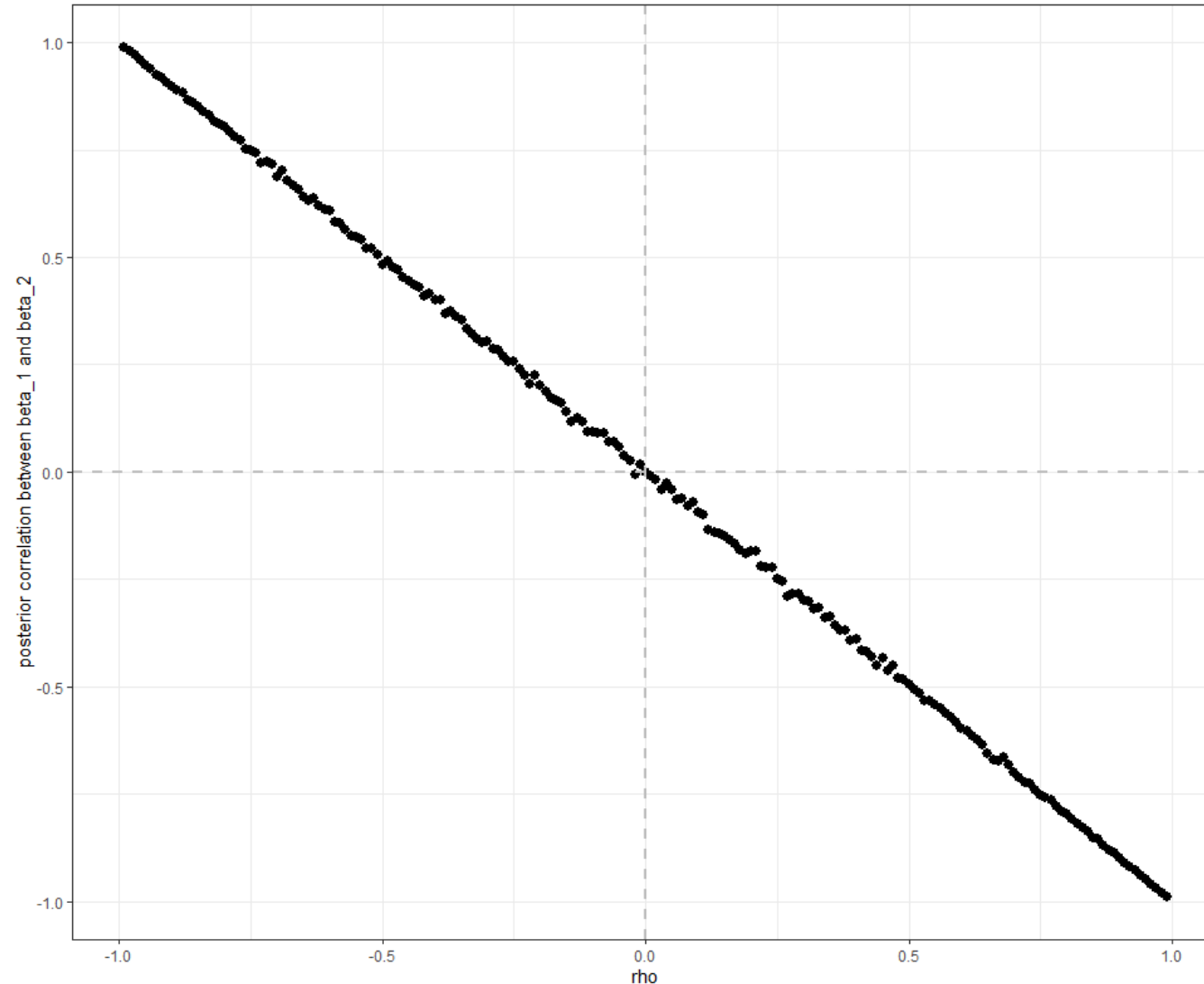
```
> ### create the design matrix
>
> X_rho_high <- model.matrix( ~ x1 + x2, xinputs_rho_high)
>
> X_rho_high %>% head()
  (Intercept)      x1      x2
1          1 -0.5176488 0.1158799
2          1  0.7375230 0.7871763
3          1 -1.6431989 -1.3113205
4          1 -0.4596897 -1.3613846
5          1  0.6683973 0.3204416
6          1  0.2035490 -0.4993101
>
> ### matrix sum of squares
> SSmat_rho_high <- t(X_rho_high) %>% X_rho_high
>
> SSmat_rho_high
  (Intercept)      x1      x2
(Intercept) 100.000000 1.670347 6.916801
x1          1.670347 94.072547 86.560880
x2          6.916801 86.560880 101.741242
>
> ### calculate the posterior covariance matrix assuming the noise is 1
> solve(SSmat_rho_high)
  (Intercept)      x1      x2
(Intercept) 0.010135793 0.002091179 -0.002468237
x1          0.002091179 0.049386178 -0.042159651
x2         -0.002468237 -0.042159651 0.045865853
>
> ### look at the posterior correlation
> cov2cor(solve(SSmat_rho_high))
  (Intercept)      x1      x2
(Intercept) 1.00000000 0.09346727 -0.1144757
x1          0.09346727 1.00000000 -0.8858283
x2         -0.11447570 -0.88582833 1.00000000
```

$\rho = 0.9$: the posterior correlation between β_1 and β_2 is strongly anti-correlated!

```
> ### create the design matrix
>
> X_rho_neg <- model.matrix( ~ x1 + x2, xinputs_rho_neg)
>
> X_rho_neg %>% head()
  (Intercept)      x1      x2
1          1 -0.1158799 -0.5176488
2          1 -0.7871763  0.7375230
3          1  1.3113205 -1.6431989
4          1  1.3613846 -0.4596897
5          1 -0.3204416  0.6683973
6          1  0.4993101  0.2035490
>
> ### matrix sum of squares
> SSmat_rho_neg <- t(X_rho_neg) %>% X_rho_neg
>
> SSmat_rho_neg
  (Intercept)      x1      x2
(Intercept) 100.000000 -6.916801 1.670347
x1         -6.916801 101.741242 -86.560880
x2          1.670347 -86.560880  94.072547
>
> ### calculate the posterior covariance matrix assuming the noise is 1
> solve(SSmat_rho_neg)
  (Intercept)      x1      x2
(Intercept) 0.010135793 0.002468237 0.002091179
x1          0.002468237 0.045865853 0.042159651
x2          0.002091179 0.042159651 0.049386178
>
> ### look at the posterior correlation
> cov2cor(solve(SSmat_rho_neg))
  (Intercept)      x1      x2
(Intercept) 1.00000000 0.11447570 0.09346727
x1          0.11447570 1.00000000 0.88582833
x2          0.09346727 0.88582833 1.00000000
```

$\rho = -0.9$: the posterior correlation between β_1 and β_2 is strongly correlated!

Examine the posterior correlation between β_1 and β_2 for various values of ρ (results generated via simulation)



We discussed that when the inputs are highly related we are actually learning the **EFFECTIVE SUM** of the two coefficients

- We are therefore learning a combination of the two!
- We are uncertain about the individual parameter values!
- To check, look at the posterior standard deviation of the β_1 and β_2 parameters for the 3 different ρ values.

```
> ### compare the posterior standard deviations
>
> sqrt(diag(solve(SSmat_rho_zero)))
(Intercept)          x1          x2
  0.1006767    0.0947553    0.1019122
>
> sqrt(diag(solve(SSmat_rho_high)))
(Intercept)          x1          x2
  0.1006767    0.2222300    0.2141631
>
> sqrt(diag(solve(SSmat_rho_neg)))
(Intercept)          x1          x2
  0.1006767    0.2141631    0.2222300
>
```

The posterior uncertainty on the slopes is **HIGHER** when the two inputs are **RELATED**!

Examine the posterior standard deviation of β_1 for various values of ρ (results generated via simulation)

